

- ① Configuration Map.
- ② Network Exposure
- ③ Traffic & Deployment Strategy.
- ④ Storage & Persistence

① Configuration Map:-

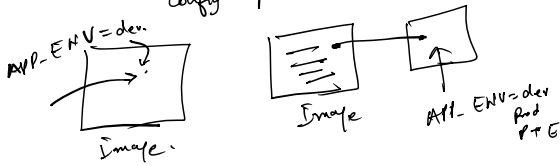
- ① Config Map Deep Dive
- ② Secret (Secure Handling)
- ③ Env. Variable vs Mounted Config
- ④ Dynamic Config. updates.

Bad Approach:-

- ① Database URL
- ② API endpoint
- ③ Password
- ④ env. name.
- ⑤ Log level.

Good Approach:-

- ① App. Image Remain Same.
- ② Config is supplied externally using Config Map & Secret.



Config Map / Secret

↓
OpenShift Pod

↓
Container will receive config as:

- ① Env. variable
- ② mounted file

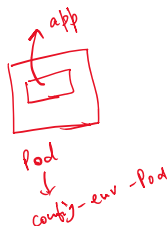
↓
App Read the Config.

① Config Map:- Non-Sensitive info.

- ② App env.
- ③ log level
- ④ Feature flag.
- ⑤ app. properties.

Config → Pod

```
cat <<EOF | oc apply -f - → yaml
apiVersion: v1
kind: Pod
metadata:
  name: config-env-pod
labels:
  app: config-env-demo
spec:
  containers:
  - name: app
    image: registry.access.redhat.com/ubi9/ubi-minimal:latest
    command: ["bin/sh", "-c"]
    args:
    - |
      while true;
      do
        echo "APP_ENV=${APP_ENV}";
        echo "LOG_LEVEL=${LOG_LEVEL}";
        echo "APP_VERSION=${APP_VERSION}";
        sleep 30;
      done
    envFrom:
    - configMapRef:
        name: app-config
    restartPolicy: Never
EOF
```



app-config
APP-ENV=dev
LOG-LEVEL=debug
APP-VERSION=2.0

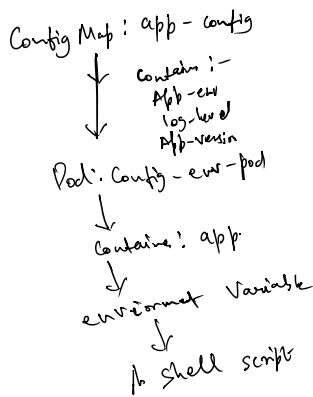
```

envFrom:
- configMapRef:
  name: app-config
  restartPolicy: Never
EOF

```

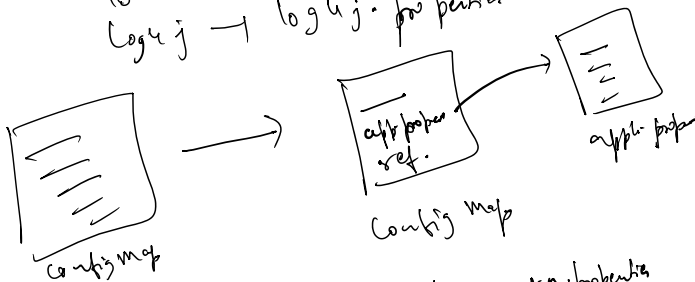
app-cont
 App Env = dev
 Log-level = debug
 App Ver = 2.0

Flow chart:-



2. Config Map as mounted file:-

Java Spring Boot → application.properties / application.yml
 NGINX → nginx.conf
 Tomcat → server.xml
 Log4j → log4j.properties

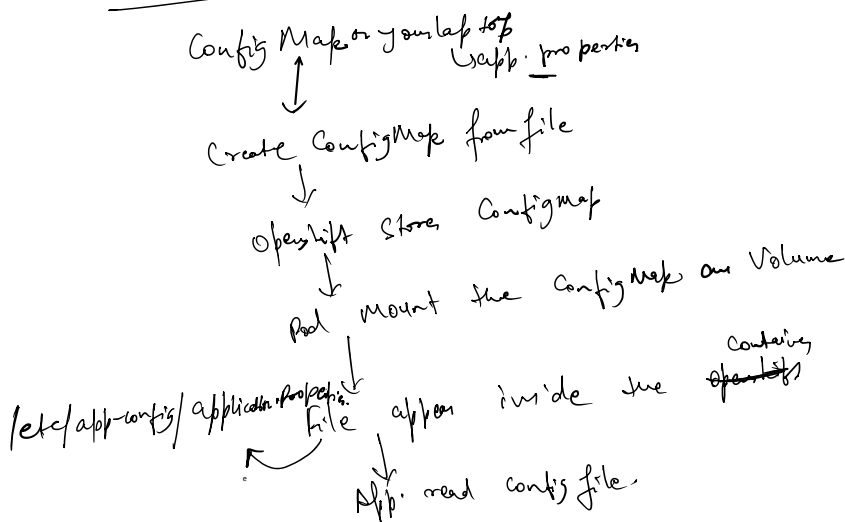


Pod → /etc/app-config/application.properties

↑

Variables

Flow diagram



```

cat <<'EOF' | oc apply -f -
apiVersion: v1

```

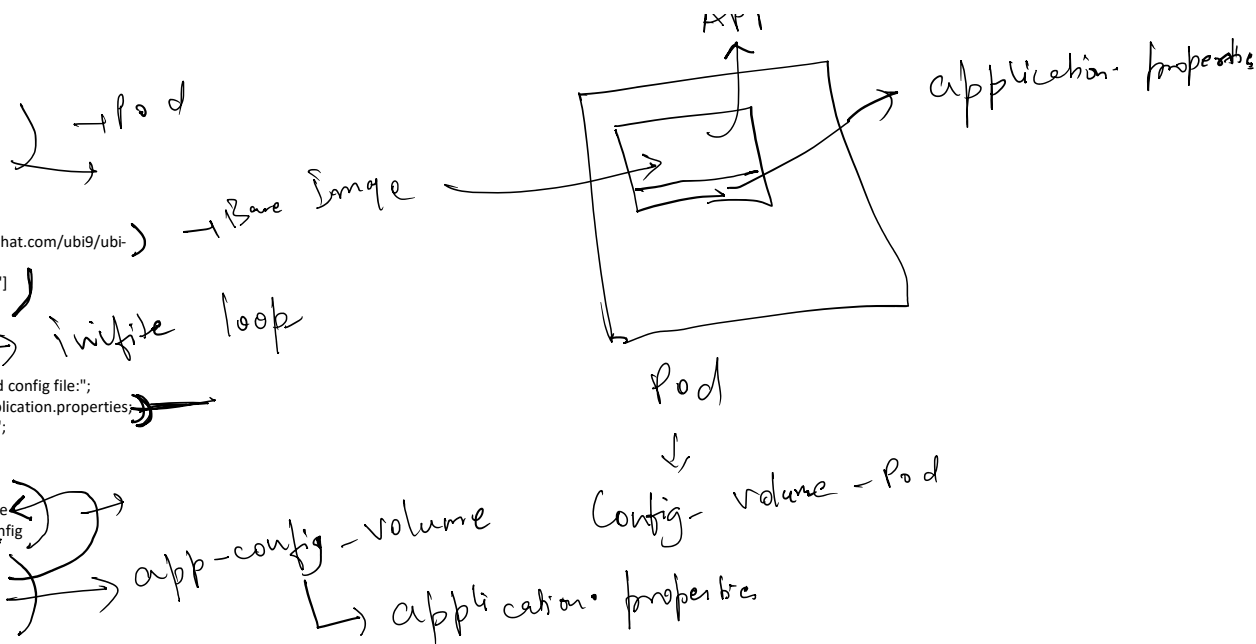
pod



```

cat <<'EOF' | oc apply -f -
apiVersion: v1
kind: Pod
metadata:
  name: config-volume-pod
spec:
  containers:
  - name: app
    image: registry.access.redhat.com/ubi9/ubi-minimal:latest
    command: ["/bin/sh", "-c"]
    args:
    - |
      while true;
      do
        echo "Reading mounted config file:";
        cat /etc/app-config/application.properties;
        echo "-----";
        sleep 30;
      done
  volumeMounts:
  - name: app-config-volume
    mountPath: /etc/app-config
  volumes:
  - name: app-config-volume
    configMap:
      name: app-file-config
    restartPolicy: Never
EOF

```

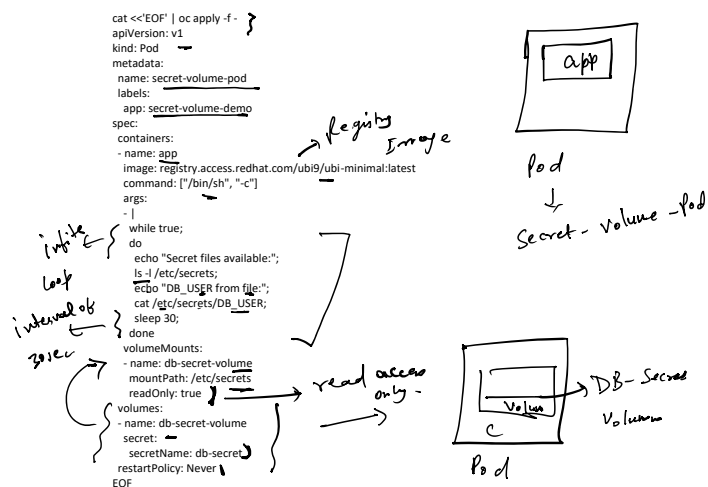


② Secure Handling:-

Secret stores sensitive data.

- DB Password
- username
- Private key
- API-Token
- Registry-credentials

* Mount file with the secret file



② Network Exposure:-

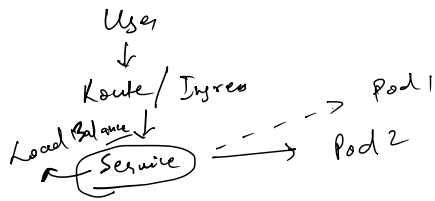
Pod → IP address
 x

Pod → ip address

X

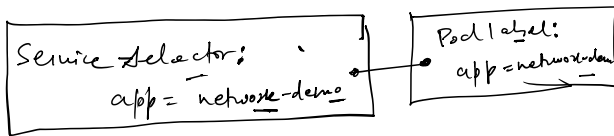
X

New Pod → New ip address



Service - Stable Network Connectivity.

Label is used to identify Pod.



Service selector: app = network-demo

Filter the Pod with label app = network-demo



Send the traffic to Matching Pod.

Load Balancer.

Service name = network-demo-svc

Service IP = 172.30.100.10

Port = 8080

Pod 1 = app = network-demo

Pod 2 = app = network-demo

Type of Service:

① Cluster IP

② Node Port

③ Load Balancer

④ External Name



① Cluster IP - Expose app inside the cluster only

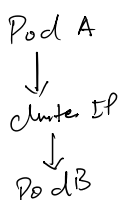
② Node Port - Expose app on node IP & Port

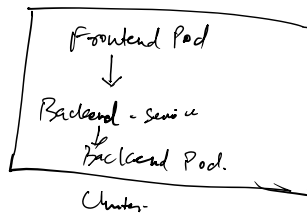
③ Load Balancer - Load Balancing

④ External Name - Map Service Name with DNS Name.

① Cluster IP Service - Default Service Type.

only inside cluster





```

cat <<'EOF' | oc apply -f -
apiVersion: v1
kind: Pod
metadata:
  name: network-demo-pod
  labels:
    app: network-demo
spec:
  containers:
  - name: app
    image: quay.io/openshift/origin-hello-
      openshift:latest
    ports:
    - containerPort: 8080
  restartPolicy: Never
EOF

```

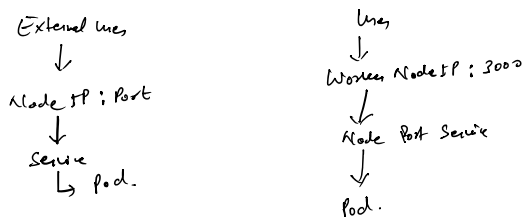
```

cat <<'EOF' | oc apply -f -
apiVersion: v1
kind: Service
metadata:
  name: network-demo-svc
spec:
  type: ClusterIP
  selector:
    app: network-demo
  ports:
  - name: http
    port: 8080
    targetPort: 8080
EOF

```

Node Port

↳ expose a service on a static port on every worker node.



Service + Route
 ↳ Host Based Access
 TLS
 OpenShift Router

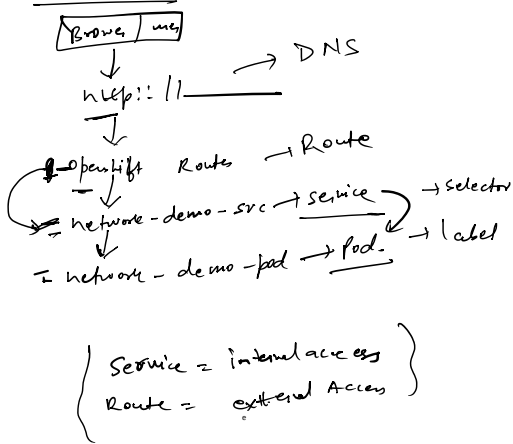
Route:- Service externally via Hostname
 External IP
 ↳ Route URL
 ↳ OpenShift Router → Service → Pod.

K8S → Ingress
 OpenShift → Route → Simply
 streamlined integration with

K8S → Ingress
 openShift → Route → simply
 integrated integration with
 openShift

- ① external Hostname
- ② HTTP/HTTPS
- ③ TLS Termination
- ④ Traffic forwarding to service
- ⑤ Traffic split
- ⑥ Blue green.

Route Diagram:-



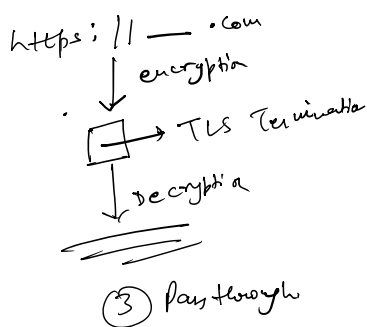
① TLS

② Storage & Persistence

- ① Persistent Volume (PV)
- ② Persistent Volume claim (PVC)
- ③ storage class.
- ④ Dynamic Provisioning
- ⑤ Stateful vs stateless apps.

① TLS:- TLS Termination

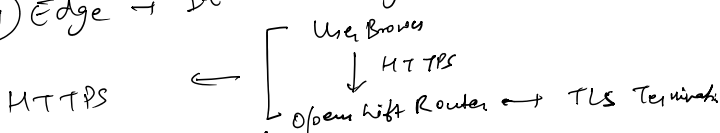
where HTTPS encryption ends.

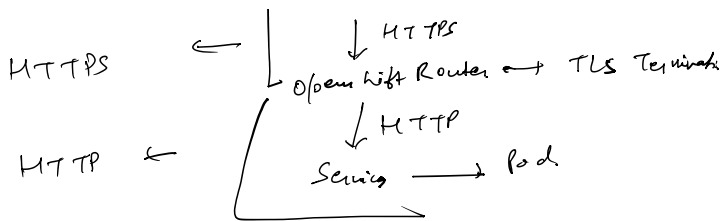


① Edge

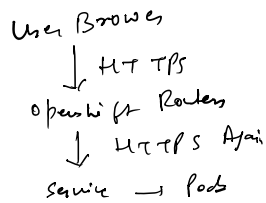
② Re-encrypt

① Edge → It will decrypt at the OpenShift Router.





② Re-encrypt :-



Use:-

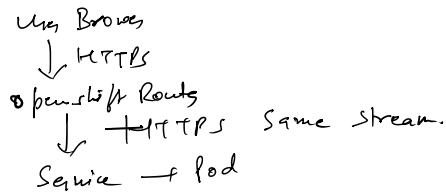
- ① Banking
- ② Hospitals/Healthcare
- ③ Payment
- ④ Compliance

Condⁿ:-

- ① Backend app must serve HTTPS
- ② Router must have backend Certificate.

③ Pass through:-

Router will forward same as it is

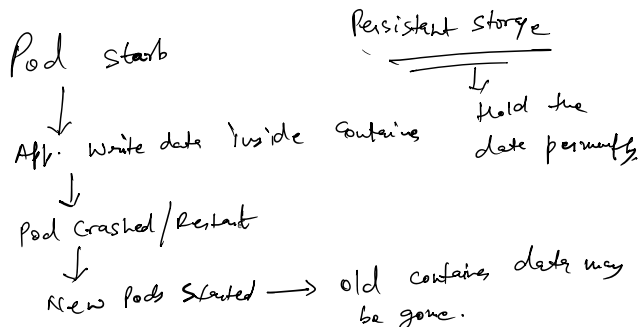


- ① Sensitive API call
- ② Client Certificate Auth.

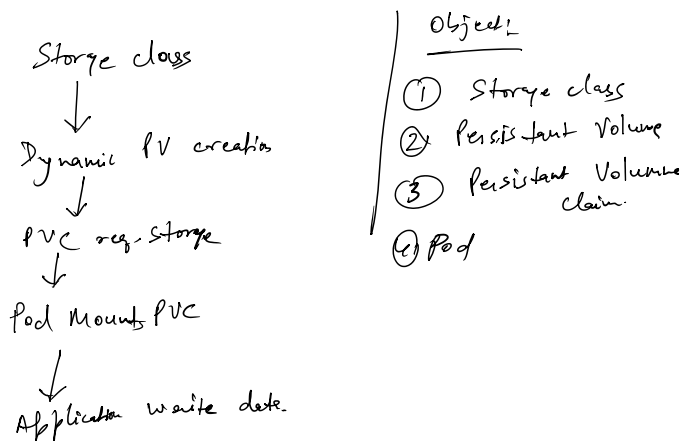
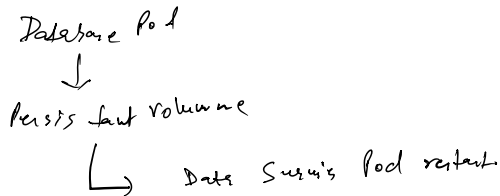
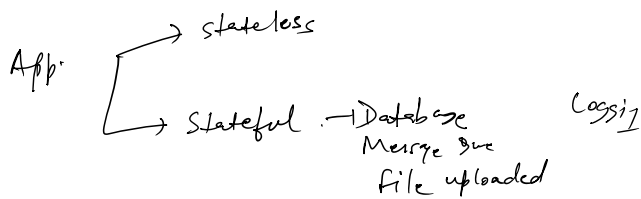
Edge → HTTPS ends at Router

Re-encrypt → HTTPS ends at Router, start again at backend

Pass through → HTTP passes through Router & ends at the app



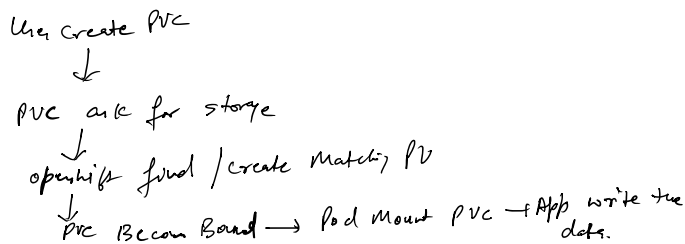
App. → stateless



① Persistent Volume (PV):- Actual storage Available to the cluster

PV = Actual storage resource

② PVC Persistent^{volume} claim:- Req. for the storage



Reclaim Policy:-

pvc deleted

↓

PV deleted

↓

Underlying storage may also deleted

Storage class:- storage profile / template.

Fast ssd
standard
gp3

Dynamic Provisioning:-

pvc created

↓

Storage class selected

PVC Created
 ↓
 Storage class selected
 ↓
 Actual storage
 ↓
 PV is automatically create
 ↓
 PVC binds to PV
 ↓
 Pod Mounts PVC

```

cat <<EOF | oc apply -f -
apiVersion: v1
kind: Pod
metadata:
  name: pvc-writer-pod
  labels:
    app: pvc-demo
spec:
  containers:
    - name: app
      image: registry.access.redhat.com/ubi9/ubi-minimal:latest
      command: ["/bin/sh", "-c"]
      args:
        - |
          echo "Writing data to persistent volume...";
          date >> /data/storage-test.txt;
          echo "Pod name: $HOSTNAME" >> /data/storage-test.txt;
          echo "Data written successfully.";
          while true;
          do
            echo "Current file content:";
            cat /data/storage-test.txt;
            sleep 30;
          done
          volumeMounts:
            - name: demo-storage
              mountPath: /data
          volumes:
            - name: demo-storage
              persistentVolumeClaim:
                claimName: demo-pvc
              restartPolicy: Never
EOF
  
```



pvc-writer-pod

Volume Mount name
 ↓
 demo-storage
 Volume : name = demo-storage.

PVC : demo-pvc
 ↓
 Bound to PV
 ↓
 Pod: pvc-writer-pod
 ↓
 Mount Path: /data
 ↓
 File: /data/storage-test.txt